

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	J Venkata Varun	Reg. No.:	192372227
Section / Batch:	AI	Date:	21/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Given an array of integers, find the maximum sum of a subsequence such that the elements are in increasing order. The subsequence must follow the original order of elements. The goal is to maximize the sum rather than the length. Use Dynamic Programming to determine the optimal solution.

Input Format:

1/1 answered

n = number of elements

arr[] = elements

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION**Coding Challenge Task**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1206

SIMATS
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

NARRAVULA NAVYA SRI
192324220RA

ASSESSMENT CO4-AT1-Coding Challenge Task

Back

Language: Python C C++ Java

QUESTIONS

Q1

Maximum Sum Increasing Subsequence

100 marks

1/1 answered

Q1 Maximum Sum Increasing Subsequence

100 marks

Given an array of integers, find the maximum sum of a subsequence such that the elements are in increasing order. The subsequence must follow the original order of elements. The goal is to maximize the sum rather than the length. Use Dynamic Programming to determine the optimal solution.

Input Format:

n = number of elements
arr[] = elements

Your Code

```
1 def max_sum_increasing_subsequence(arr):  
2     n = len(arr)  
3     dp = arr[1]  
4  
5     for i in range(n):  
6         for j in range(i):  
7             if arr[i] < arr[j]:  
8                 dp[i] = max(dp[i], dp[j] + arr[i])  
9  
10    return max(dp) if n > 0 else 0  
11 n = int(input().strip())  
12 arr = list(map(int, input().split()))  
13 print(max_sum_increasing_subsequence(arr))
```

Submitted

Input

```
5  
1 2 2 3 4
```

Output

```
10
```